

La Cancha Inteligente

Desafío 3 - Copa de Algoritmia y Programación UADE 2026

Carreras de Informática y Sistemas

Integrantes

Lucas Abad

Thiago Albornoz

Valentino Sarniguette

Gaston Trezeguet

Valentin Zaccari

1. Descripción general

El proyecto La Cancha Inteligente consiste en un simulador táctico desarrollado en Python. El programa representa una cancha de fútbol mediante una matriz de 40 filas por 60 columnas, donde se pueden cargar jugadores, obstáculos, árbitro, sombra del árbitro y distintas situaciones de juego.

El sistema permite analizar una jugada a partir de la ubicación de los jugadores en la cancha. Entre sus funcionalidades principales se encuentran el movimiento de jugadores, el cálculo de distancia a la pelota, la detección de pases posibles y la verificación de camino libre al arco.

Además de cumplir con los requerimientos principales del desafío, el programa incorpora un menú interactivo, validaciones de datos, escenario de prueba automático, reinicio de cancha, borrado de jugadores y movimiento aleatorio del árbitro cerca de la jugada.

2. Requisitos

Para ejecutar el programa se necesita Python 3 instalado. No se requieren librerías externas. El programa utiliza estructuras básicas de Python: listas, diccionarios, funciones, condicionales y ciclos.

La única librería importada es random, que pertenece a la librería estándar de Python y se utiliza para mover aleatoriamente al árbitro cerca de la jugada.

3. Cómo ejecutar el programa

El archivo principal debe ejecutarse desde una terminal o desde un entorno como Visual Studio Code.

```
python desafio3.py
```

4. Modo de uso rápido

- Ejecutar el archivo principal del programa.
- Desde el menú principal, ingresar a la opción 2 - Ejecutar cancha inteligente.
- Dentro del simulador, cargar una jugada manualmente agregando jugadores y obstáculos, o usar la opción 7 para cargar el escenario de prueba.
- Para analizar la jugada, elegir la opción 4 - Analizar jugada. En ese momento el árbitro se mueve aleatoriamente cerca de la pelota y su sombra queda marcada como zona bloqueada.
- Para iniciar otra jugada desde cero, usar la opción 8 - Reiniciar cancha.
- Para salir, volver al menú principal con 0 - Volver y luego elegir 0 - Salir.

5. Estructura general del programa

El programa está organizado en bloques funcionales para que cada parte tenga una responsabilidad clara.

Bloque	Responsabilidad
1	Constantes y creación de la cancha
2	Validaciones básicas
3	Creación y posicionamiento de jugadores
4	Obstáculos
4.1	Árbitro y sombra
5	Movimiento de jugadores
6	Distancia a la pelota
7	Pases posibles
8	Camino libre al arco
9	Funciones de ingreso de datos
10	Opciones del submenú
11	Escenario de prueba
12	Menú
13	Inicio del programa

6. Representación de la cancha

La cancha se representa mediante una matriz de 40 x 60. Las filas válidas van de 0 a 39 y las columnas válidas van de 0 a 59.

Símbolo	Significado
.	Celda vacía
A	Jugador de Argentina
B	Jugador de Brasil
X	Obstáculo, árbitro o sombra del árbitro

La pelota no ocupa una celda propia. Siempre está asociada a un jugador y comparte su posición.

7. Representación de los jugadores

Cada jugador se representa mediante un diccionario de Python. Los jugadores se almacenan dentro de un diccionario general llamado jugadores, donde cada clave es un identificador numérico.

```
{
    "nombre": "Messi",
    "equipo": "A",
    "fila": 20,
    "columna": 20,
    "rol": "delantero",
    "tiene_pelota": "S"
}
```

8. Equipos y roles válidos

Equipo	Significado
A	Argentina
B	Brasil

Los roles válidos son arquero, defensor, mediocampista y delantero. Si el usuario ingresa un equipo o rol inválido, el programa muestra un mensaje de error y vuelve a pedir el dato.

9. Funcionalidades principales

9.1 Crear cancha

La función crear_cancha() genera una matriz de 40 filas y 60 columnas. Cada celda se inicializa con un punto.

9.2 Agregar jugadores

Permite cargar nombre, equipo, fila, columna, rol y posesión de pelota. Valida equipo, rol, posición, celda ocupada y pelota única.

9.3 Agregar obstáculos

Permite colocar un obstáculo X en una celda libre dentro de los límites de la cancha.

9.4 Árbitro y sombra

El programa crea una estructura de árbitro que almacena sus posiciones bloqueadas. Al analizar una jugada, el árbitro se mueve aleatoriamente cerca del jugador con pelota y se marca su celda y sombra como X. Esas celdas bloquean pases y camino al arco.

9.5 Mover jugadores

Permite mover un jugador arriba, abajo, izquierda o derecha. Valida dirección, límites y celda destino libre.

9.6 Calcular distancia a la pelota

Calcula la distancia Manhattan entre cada jugador y el portador de la pelota.

9.7 Detectar pases posibles

Analiza si el jugador con pelota puede pasar a un compañero en la misma fila o columna, sin rivales ni obstáculos entre ellos. Las celdas X, incluyendo árbitro y sombra, bloquean el pase.

9.8 Detectar camino libre al arco

Analiza si un delantero está en mitad ofensiva y no tiene rivales u obstáculos en su misma fila hacia el arco rival. Las zonas X también bloquean este camino.

9.9 Listar jugadores

Muestra ID, nombre, equipo, fila, columna, rol y posesión de pelota.

9.10 Borrar jugador

Elimina un jugador por ID, libera su celda y avisa si tenía pelota.

9.11 Reiniciar cancha

Crea una cancha nueva y elimina jugadores, obstáculos y zonas del árbitro.

9.12 Mostrar cancha

Imprime la matriz completa, avisando previamente que la salida es extensa.

9.13 Escenario de prueba automático

Carga una jugada predefinida para evaluar rápidamente distancias, pases, camino al arco y el movimiento del árbitro.

Fórmula de distancia Manhattan:

```
abs(fila_jugador - fila_pelota) + abs(columna_jugador - columna_pelota)
```

10. Menú del sistema

Menú principal:

```
===== MENU PRINCIPAL =====
1 - Modo de uso
2 - Ejecutar cancha inteligente
3 - Acerca de
0 - Salir
```

Submenú del simulador:

```
===== SIMULADOR DE CANCHA =====
1 - Agregar jugador
2 - Agregar obstaculo
3 - Mover jugador
4 - Analizar jugada
5 - Listar jugadores
6 - Mostrar cancha
7 - Cargar escenario de prueba
8 - Reiniciar cancha
9 - Borrar jugador
0 - Volver
```

11. Validaciones implementadas

- Equipo válido: solo A o B.
- Rol válido: arquero, defensor, mediocampista o delantero.
- Posición dentro de la cancha: filas de 0 a 39 y columnas de 0 a 59.

- Celda libre antes de agregar jugador u obstáculo.
- Una sola pelota en juego.
- Dirección válida para movimiento.
- Ingreso numérico controlado con try/except.
- Nombre de jugador no vacío.
- Cancelación de carga escribiendo NO.
- Bloqueo de movimiento fuera de cancha o hacia celda ocupada.
- Control de análisis si no hay suficientes jugadores o no hay pelota.
- Liberación de celda al borrar un jugador.
- Limpieza de la ubicación anterior del árbitro y su sombra antes de moverlo nuevamente.
- Validación de que el árbitro se ubique en una celda libre cercana a la pelota.

12. Casos de prueba recomendados

Caso	Acción	Resultado esperado
1	Cargar escenario de prueba	Jugadores, obstáculos y situación táctica listos para analizar
2	Agregar jugador válido	Se agrega y se marca en la cancha
3	Superponer jugador	Se rechaza celda ocupada
4	Duplicar pelota	Se evita segunda pelota
5	Mover fuera de cancha	Movimiento bloqueado
6	Mover a celda ocupada	Movimiento bloqueado
7	Agregar obstáculo	Celda marcada con X
8	Pase bloqueado por rival	Se informa bloqueo
9	Pase posible	Se informa pase disponible
10	Camino bloqueado	Delantero sin camino libre
11	Árbitro y sombra	Se ubican como X cerca de la jugada al analizar
12	Borrar jugador	Se libera la celda
13	Reiniciar cancha	Cancha vuelve a estar vacía

Caso de prueba rápido recomendado:

- 2 - Ejecutar cancha inteligente
- 7 - Cargar escenario de prueba
- 5 - Listar jugadores
- 4 - Analizar jugada
- 6 - Mostrar cancha
- 9 - Borrar jugador

```
8 - Reiniciar cancha
0 - Volver
0 - Salir
```

13. Decisiones de diseño

Uso de matriz

Se eligió representar la cancha como una matriz porque la consigna trabaja con posiciones de fila y columna.

Uso de diccionarios para jugadores

Cada jugador se representa como diccionario. Además, un diccionario general con ID numérico facilita la selección desde el menú.

Pelota como atributo del jugador

La pelota se representa con el campo tiene_pelota y no como celda independiente.

Árbitro como estructura separada

El árbitro se representa con un diccionario que guarda sus posiciones bloqueadas. Esto permite limpiar su ubicación anterior y moverlo nuevamente sin afectar jugadores ni obstáculos cargados por el usuario.

Uso de random

Se utiliza random.randint() para ubicar al árbitro cerca de la pelota dentro de un rango de 3 celdas. random pertenece a la librería estándar de Python.

Menú interactivo

El menú mejora la experiencia del usuario y facilita la evaluación manual.

Escenario de prueba

Permite verificar rápidamente las funcionalidades principales sin cargar datos manualmente.

Borrado y reinicio

Permiten corregir errores de carga y probar nuevas jugadas de manera cómoda.

14. Restricciones y aclaraciones

- El programa no simula un partido completo. Representa una situación puntual de juego sobre una cancha matricial.
- El sistema detecta pases posibles, pero no ejecuta el pase ni cambia la posesión de la pelota. Esta decisión se tomó porque el desafío solicita detectar si existen pases posibles, no simular una acción completa de pase.
- La pelota no ocupa una celda propia en la matriz. Está asociada al jugador que la posee.
- Las celdas X representan obstáculos generales y también las posiciones del árbitro y su sombra.
- El árbitro se mueve al momento de analizar la jugada. Por lo tanto, el resultado de los pases posibles puede variar si el árbitro queda bloqueando una línea de pase.
- El escenario de prueba reemplaza la cancha actual por una cancha nueva con datos predefinidos.

15. Conclusión

El programa desarrollado cumple con los requerimientos principales del Desafío 3 mediante una solución modular, validada e interactiva.

La cancha se representa mediante una matriz de 40 x 60, los jugadores mediante diccionarios y las funcionalidades se organizan en funciones específicas. El sistema permite cargar y mover jugadores, agregar obstáculos, calcular distancias, detectar pases posibles, analizar camino libre al arco y considerar al árbitro y su sombra como zonas bloqueadas.

Además, se agregaron mejoras de experiencia de usuario como menú interactivo, escenario de prueba, reinicio de cancha y borrado de jugadores. Estas mejoras permiten una evaluación más clara y facilitan el uso del sistema sin modificar el código fuente.